

A Graph-Based Approach to Optimal Path Search Using Probability Heuristics

Béla Romeo Bernasconi
Andrés Santiago Martínez Hernández
Sébastien Pierret
Ali Abdel Ghaffar

May 22, 2026

Contents

1	Introduction	2
2	Data acquisition & preprocessing	3
3	Methodology	4
3.1	Graph Construction	4
3.2	Network Exploration and macro-structure	5
3.2.1	Network Density	5
3.3	Positional Favorability & Expected Value	9
3.4	Community Detection: Strategic Subtree Partitioning	10
3.5	Depth-Limited Path Scoring	11
3.5.1	Algorithm Design	11
3.5.2	Discussion of Design Choices	11
3.6	Path Analysis	11
3.7	Regression Model	12
3.8	Ablation Study	13
3.9	Regression Results	15
3.10	Stockfish comparison	15
4	Network Visualization	17
5	Conclusion	18
6	References	18

1 Introduction

Regardless of age or background, chess ranks among the most widely played and enduring board games in human history, and its complexity is staggering: Claude E. Shannon’s conservative lower bound places the number of possible chess games at approximately 10^{120} (Shannon’s number), while more ambitious estimates by Godfrey H. Hardy reach as high as $10^{10^{50}}$. Beyond its status as a game, chess sits at a natural intersection of mathematics, computer science, and graph theory. Existing graph-theoretic treatments of chess, however, tend to operate at a small scale. The Knight’s Tour is the canonical example, where the board itself forms the vertex set and the problem reduces to finding a Hamiltonian path. While such formulations can be scaled up, the resulting network patterns remain structurally rigid and self-similar. They do not capture anything about how chess is actually played.

What is perhaps surprising is how rarely chess has been approached as a large-scale network. The representational possibilities are rich, yet the literature has not explored this space in earnest. The reason is partly computational: Building and analyzing a graph over millions of games and possible combinations requires immense resources. The deeper reason we suggest is historical. Since Shannon’s foundational 1950 paper, computational chess research has been dominated by game-tree search oriented toward playing chess optimally. This paradigm is powerful but narrow: it treats chess as an optimization problem and discards everything about the structure of how humans actually navigate the position space.

This project departs from that tradition. We model chess theory itself as a directed graph, constructed from Lichess game data, in which nodes represent board positions and weighted edges represent move transitions scaled by frequency. Chess is naturally a sequential decision process, each move transforms one board state into another, and representing those transitions as a network makes it possible to study the opening phase using graph-analytic tools rather than move lists or engine output alone. The graph is built from real games at or above 1000 Elo with a minimum time control of five minutes, and covers the first twenty plies. Plies are half-moves, so white plays, black plays and when it’s white’s turn again we are at ply three.

With the graph constructed, the central question becomes: which paths through it are most valuable for a human player to know? This is not a question engines answer well, because engines evaluate positions in isolation and at depth, they do not consider whether a sequence of moves is consistently navigable across thousands of games played by real people under time pressure. To address this, we implement two complementary algorithms. A custom depth-first search scores paths by combining transition probability with empirical outcome rates, identifying lines that are both frequently played and statistically successful. A Random Forest regression model, trained on structural path features derived from the graph, learns to predict path quality from topology alone, capturing patterns such as bottleneck transitions and positional branching that the DFS score does not directly encode. Together, these methods surface candidate opening lines from the graph without any hard-coded chess knowledge.

A key motivation for this approach is that engine evaluation and human playability are not the same thing. Stockfish may assess a position as equal, yet one side can be substantially easier to play: more viable continuations exist, plans are clearer, and the margin for error is wider. At non-master level, drawn positions produce decisive results with regularity precisely because one side demands greater precision than most players can consistently deliver. Our graph captures this empirically. Rather than identifying the theoretically optimal move, it reflects which positions and transitions humans actually navigate successfully and at what cost. This makes it a complementary lens to engine analysis, and in some respects a more practically relevant one: preparation built purely on engine output can lead players into positions that are defensible in theory but consistently losing in practice.

The analysis therefore combines three perspectives: structural graph measures derived from the network topology, empirical outcome data aggregated across millions of games, and direct comparison with Stockfish evaluation. To quantify the gap between human navigation and engine optimality, we introduce a graph-based danger score, measuring at each position how sharply outcomes deteriorate when the recommended move is not found, and compare our path rankings against Stockfish centipawn evaluations using Spearman rank correlation and NDCG. Possible divergence between the two is not noise. It is the signal.

2 Data acquisition & preprocessing

Data sources: We used external sources, namely the chess and Stockfish APIs, as well as our own constructed dataset. The dataset consists of rated standard chess games from the Lichess open database. Games are filtered before graph construction using player rating and time-control thresholds. For each retained game, the parser reads the opening sequence up to a fixed depth and records the observed board-state transitions.

We first filter out all games that do not meet both of the following requirements.

Time Control: Games played with a time control of 5 minutes or below were excluded. The rationale behind this is that time control heavily influences playing style. In lower time controls, the speed at which you play your moves matters considerably more than the quality of the move you play. To balance move quality against sample size (the majority of chess games played online are played at Blitz and the lower end of Rapid time controls, e.g. 10 minutes), we decided on a lower bound of 5 minutes, which corresponds to the upper bound of the Blitz category.

Rating: The second requirement was that both players have an Elo rating of 1000 or above. Our goal was not to study master-level play exclusively, but rather to capture general human approaches to chess. However, a lower bound was considered necessary, as very low-rated players tend to make moves without strategic intent, often lacking a coherent plan or positional understanding of the game, and would therefore introduce noise into our data. We concluded that a rating of 1000 was a sensible cutoff, eliminating roughly the bottom fifth of players on Lichess. Above this threshold, players can reasonably be expected to have some intermediate chess understanding beyond basic piece movement.

Key processing efficiency implementations: **Maximum Move Depth:** To increase the speed of parsing games, we implemented a maximum move depth. Previously, every game passing the filters would be parsed in full, which unnecessarily slowed down processing. This was unnecessary, since there is little point in processing all moves of an 80-move game if positions occurring beyond a certain depth will never accumulate enough visits across the database to survive the pruning step. This consideration depends on the number of games processed and the desired graph size, but deeper parsing also slows processing. The decision was confirmed by inspecting the longest paths in earlier iterations, where the deepest surviving path spanned only 13 nodes. Based on this, a depth limit of 20 plies was adopted.

Custom PGN Tokenizer: To further accelerate processing, we replaced the chess library’s built-in `chess.pgn.read_game()` function with a custom PGN tokenizer. The library function performs a number of operations beyond simple move extraction. Most notably, it constructs a full game tree in memory and validates the legality of each move as it parses. In our case, this is unnecessary: the Lichess games from 2015-05 have already been checked for illegal moves, since illegal moves cannot be made online. Our tokenizer reads raw move tokens directly from the PGN text, bypassing tree construction and legality checks entirely. The chess library is still used where needed: for Zobrist hashing, which allows board positions to be represented as

compact integer keys rather than verbose FEN strings, and for reconstructing a visual board representation when required.

With these implementations, we were able to process our 2.1 million chess games within 23 minutes.

Table 1: Dataset and preprocessing parameters.

Parameter	Value
Source	Lichess rated standard games
Month	2013-02 / 2015-05
Minimum Elo	1000
Minimum main time	300 s
Maximum move depth	20 plies
Position representation	FEN with Zobrist hash key

Resulting data: After parsing approximately 2.1 million games from the Lichess May 2015 database, around 970,000 games passed the filtering criteria, yielding approximately 6.6 million unique board positions. After pruning — with a maximum of 10,000 candidate nodes, a minimum edge probability of 0.02, and a minimum visit count of 100 — the final graph contains 6,196 nodes and approximately 9,013 edges. The pruning parameters can be freely adjusted without re-parsing, as the raw parsed data is serialised to a checkpoint file between the two steps, allowing the pruning step to be re-run independently. While the original target was 10,000 nodes, scaling the dataset from 100,000 to 2.1 million games yielded only around 400 additional nodes under identical filtering and pruning criteria, suggesting that the graph had largely saturated at this level.

Data representation and structure: The data is stored as a JSON file structured as a nested adjacency list. This representation was chosen for two reasons. First, it maps naturally onto the directed graph structure of the data. Second, and more importantly, it preserves traversal history: The same board position can be reached via many different move sequences and at different plies in a game. By structuring the data as an adjacency list, positions can be topologically ordered, ensuring that the sequence in which a position was reached is retained.

3 Methodology

3.1 Graph Construction

Each unique board position is represented as a node. For every legal move observed in the filtered games, a directed edge is added from the current position to the next position. The edge count stores how often that transition appears, and the edge probability is computed from the transition frequency among all outgoing edges of the source position.

The project uses the python-chess library to parse legal moves and to represent board states as FEN strings. Each node is keyed by a Zobrist hash and stores its FEN representation, total visit count, and aggregate outcomes. After loading the raw graph data, nodes are sorted by total visits so that the most common positions can be inspected first.

Chess Superstructure: Opening Transitions
(Red: Starting Position | Blue: First Legal Moves)

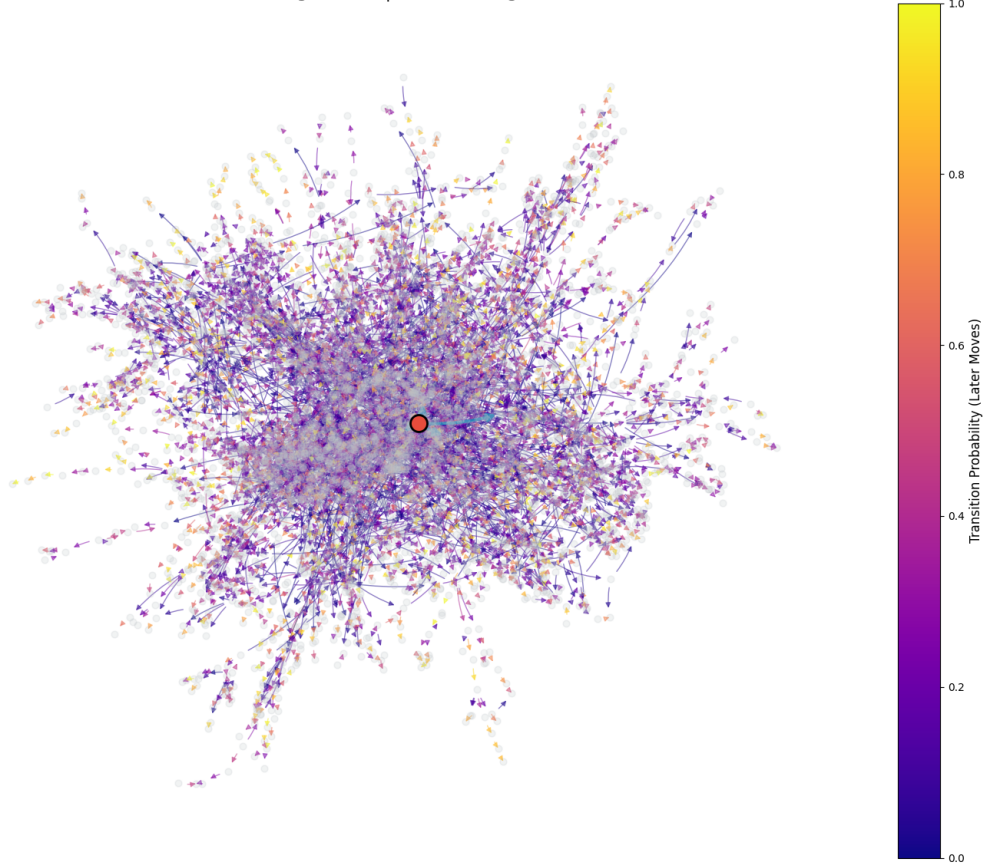


Figure 1: General visualization of the graph. Each node represents one of the approximately 6,000 board states in the directed graph. Arrow color indicates transition probability, the red node marks the starting position, and the blue arrows mark the next-layer edges.

The result is a graph that encodes all 6196 nodes and 9013 edges with specific attributes such as the fen structure for the nodes and the probability and outcomes for the edges. While this graphic provides us with an early intuition of potential paths to follow, a much more rigorous analysis of these potential paths is required.

3.2 Network Exploration and macro-structure

The network exploration phase characterises the structural properties of the opening graph before any path analysis is performed. Four measures are computed: density, PageRank, betweenness centrality, and out-degree centrality. Together they provide a multi-scale picture of the graph — from its global connectivity to the local importance of individual positions — and each maps onto a concrete chess concept that motivates its inclusion.

3.2.1 Network Density

Network density Δ is defined as the ratio of observed edges to the maximum number of edges possible in a directed graph of the same size: $\Delta = |E| / (|V| \cdot (|V| - 1))$. For the full opening graph ($|V| = 6,196$), the measured density is $\Delta = 0.00023$, indicating that fewer than 0.025% of all possible directed connections between positions are realised in the dataset.

This result states a structural property of chess itself. From any given board position, the

number of legal moves is typically between 20 and 40, a small fraction of the 6,195 other nodes in the graph. Furthermore, the opening graph is acyclic in practice: players do not cycle through the same positions repeatedly in the opening phase, so the graph grows as a directed tree with limited cross-connections from transpositions. A density value close to zero is therefore the expected and correct result, confirming that the graph faithfully represents the sequential, branching nature of chess opening play.

This sparsity has a direct methodological consequence: it motivates the use of depth-limited local traversal — specifically the beam-search DFS — as the primary analysis tool, rather than global shortest-path algorithms that assume denser connectivity.

PageRank To measure transpositional centrality: a position scores highly not because many games visit it directly, but because it is reachable from many other important positions. That way we capture the idea that multiple opening lines funnel into the same middlegame structure regardless of the move order used to get there.

PageRank is computed on the full weighted graph, using transition probabilities as edge weights. The top-ranked position achieves a score of 0.00250 and corresponds to the following FEN:

```
r1bq1rk1/ppp2pp1/2np1n1p/2b1p3/2B1P3/P1NP1N1P/1PP2PP1/R1BQ1RK1 b - - 0 8
```

This is a symmetric, double-castled middlegame structure with opposing bishops on c4 and c5, knights developed to f6 and c6, and both kings safely tucked away, a configuration characteristic of the Giuoco Piano and Four Knights families. Crucially, this position ranks only 4,530th by raw visit count, with just 156 games reaching it directly. The gap between its visit rank and its PageRank rank is the key insight: this is not a popular position, it is a structurally inevitable one. Many distinct opening sequences converge toward this middlegame configuration through different transposition paths, making it a hub in the graph even though no single path to it is particularly common.

This result illustrates the value of PageRank over raw visit frequency as an analytical lens: visit count identifies the most-travelled positions, while PageRank identifies the positions that the opening graph’s structure consistently points toward.

Betweenness Centrality Betweenness centrality measures how often a node lies on the shortest path between two other nodes in the graph (bridge behavior). In the context of the opening network, a high betweenness score identifies positions that act as unavoidable bottlenecks: nodes that a large proportion of paths must pass through to connect earlier opening moves to later middlegame structures.

Betweenness centrality is computed on the top-300 most-visited subgraph, given its $O(|V||E|)$ computational cost on the full graph. The top-ranked position achieves a score of 0.00012 and corresponds to:

```
r1bqkbnr/pppp1ppp/2n5/4p3/4P3/5N2/PPPP1PPP/RNBQKB1R w KQkq - 2 3
```

This is the position after 1.e4 e5 2.Nf3 Nc6 — one of the most fundamental tabiya positions in chess, reached at the start of every Two Knights, Four Knights, Spanish, and Italian game. With 102,210 visits, it is among the highest-traffic nodes in the dataset, and its betweenness score confirms what chess intuition would suggest: this position is not just popular, it is structurally inescapable for any game that opens 1.e4 e5.

The contrast with the PageRank result is instructive. The top PageRank node was a rare position (156 visits, rank 4,530) that many important paths pointed toward without passing through it

frequently. The top betweenness node is the opposite: an extremely common position that sits at the junction of many independent path pairs, acting as a bridge between the initial moves and the full diversity of $1.e4\ e5$ middlegames. The two metrics are therefore complementary: PageRank finds transpositional destinations, betweenness finds unavoidable waypoints.

Out-Degree Centrality Out-degree centrality quantifies the volume of discrete outward transitions originating from a specific node. Within the opening network, this metric serves as a proxy for maximum practical complexity, highlighting positions where the repertoire of viable human continuations is most extensive and the subsequent strategic paths are most varied.

The node exhibiting the highest out-degree in the dataset possesses 15 outgoing edges and is represented by the FEN:

rnbqkbnr/pppp1ppp/8/4p3/8/4P3/PPPP1PPP/RNBQKBNR w KQkq - 0 2

This board state arises following $1.e3\ e5$. Despite a relatively modest traffic volume of 8,767 visits, this non-committal approach for White yields more distinct branching paths than any higher-traffic alternative. By refraining from an early central confrontation, White maintains maximum optionality, allowing the game to fluidly transpose into varied systems like the French Defense or Italian Game. Unlike the structural inevitability found in high-betweenness nodes, this position identifies where the opening tree experiences its most significant expansion.

This finding completes our multi-scale structural analysis. While PageRank and betweenness centrality identify transpositional hubs and unavoidable waypoints, out-degree centrality isolates branching complexity (i.e., the specific junctions where human preparation is most heavily tested by the sheer diversity of available responses).

We propose the following representation of the top nodes that correspond to these metrics.

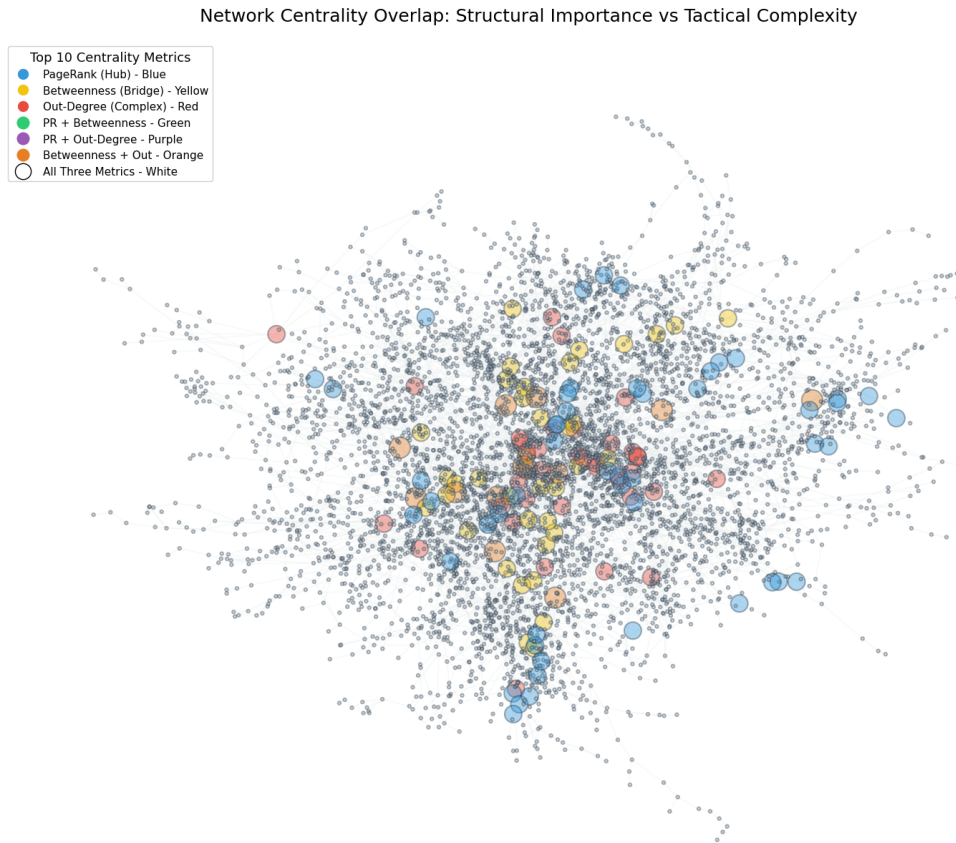


Figure 2: Centrality overlap of nodes. The graph highlights the top 50 nodes by PageRank, betweenness centrality, and out-degree centrality; combined colors indicate nodes that rank highly on more than one metric.

3.3 Positional Favorability & Expected Value

To implement a graph-analytic technique capable of surfacing meaningful insights within our specific domain, we propose a custom path scoring method. By utilizing a modified depth-limited depth-first search (DFS) algorithm, we leverage one of the most effective information diffusion frameworks to produce an outcome-weighted ranking of game sequences.

Each node stores game outcomes in the order white wins, black wins, and draws. The expected value of a position is computed as:

$$EV(v) = \frac{W(v) - B(v)}{W(v) + B(v) + D(v)}$$

Here $W(v)$, $B(v)$, and $D(v)$ are the number of white wins, black wins, and draws observed after visiting position v . Positive values favor White, negative values favor Black, and values close to zero indicate balanced outcomes.

To enrich the search process, we implement a custom expected value function derived from empirical position-level outcomes, specifically the aggregate counts of wins, losses, and draws recorded at each node. Following Equation 1, this metric provides a normalized heuristic used to guide a modified DFS algorithm, scoring potential transitions by the product of their observed probability and the expected value of the resulting board state.

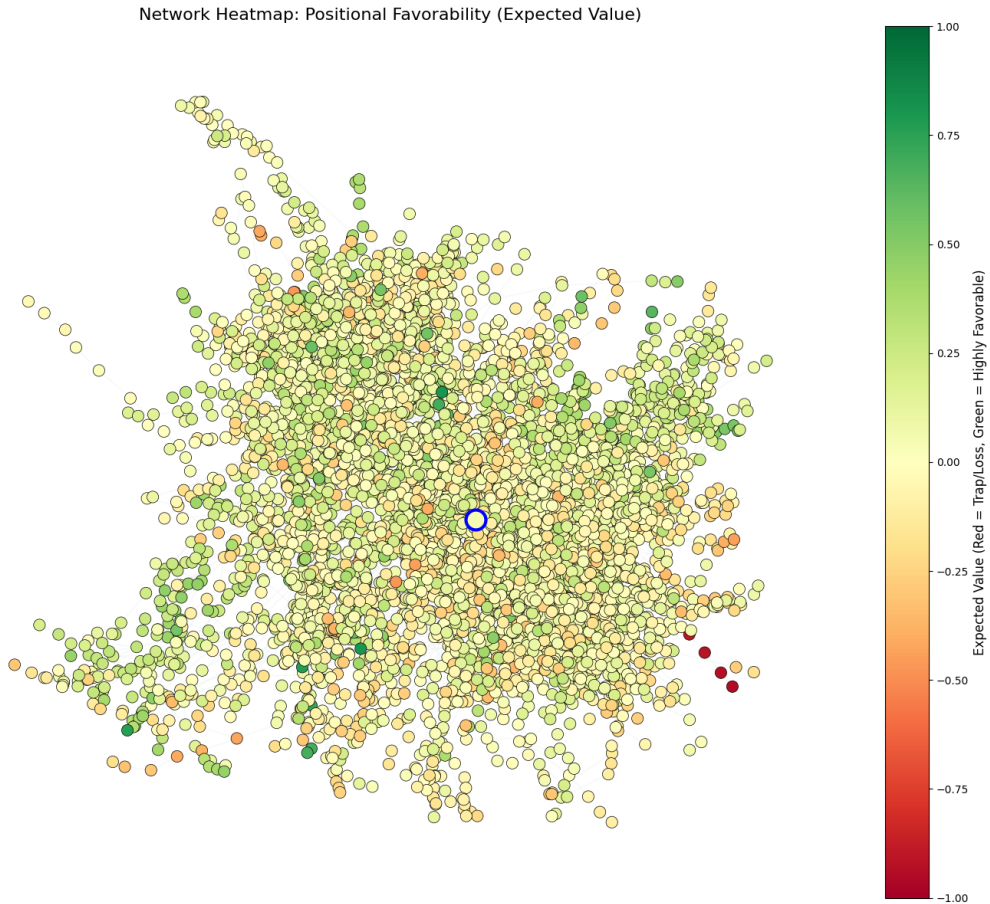


Figure 3: Mapping of expected value across the graph. The color map ranges from -1 to 1, where 1 is highly favorable for White, 0 is neutral, and -1 is favorable for Black.

3.4 Community Detection: Strategic Subtree Partitioning

Traditional community detection algorithms, such as Greedy Modularity, are structurally unsuitable for this domain because they ignore the chronological directionality of move sequences, effectively implying temporal reversibility. To segment the network in a manner consistent with the flow of a chess game, we implement Directed Subtree Partitioning. By initiating a directed Breadth-First Search (BFS) from the highest-traffic root transitions, the graph is successfully partitioned into distinct "Opening Families"—clusters such as the e4 and d4 systems that represent isolated strategic regimes.

To quantify the structural and strategic properties of these clusters, we evaluate three key metrics:

Retention Rate (Intra-Cluster Transition Probability): Quantifies the probability that a transition remains within the same Opening Family. High retention suggests a "forcing" system that restricts transpositional optionality for the opponent.

Cluster Purity (EV Variance): Measures the stability of the heuristic outcome signal within the subtree. Low variance characterizes stable positional lines, while high variance identifies volatile tactical battlegrounds where the margin for error is narrow.

Branching Complexity (Average Out-Degree): Provides a structural proxy for the density of viable continuations, distinguishing between "open" complexity and the restrictive bottlenecks of theoretical main lines.

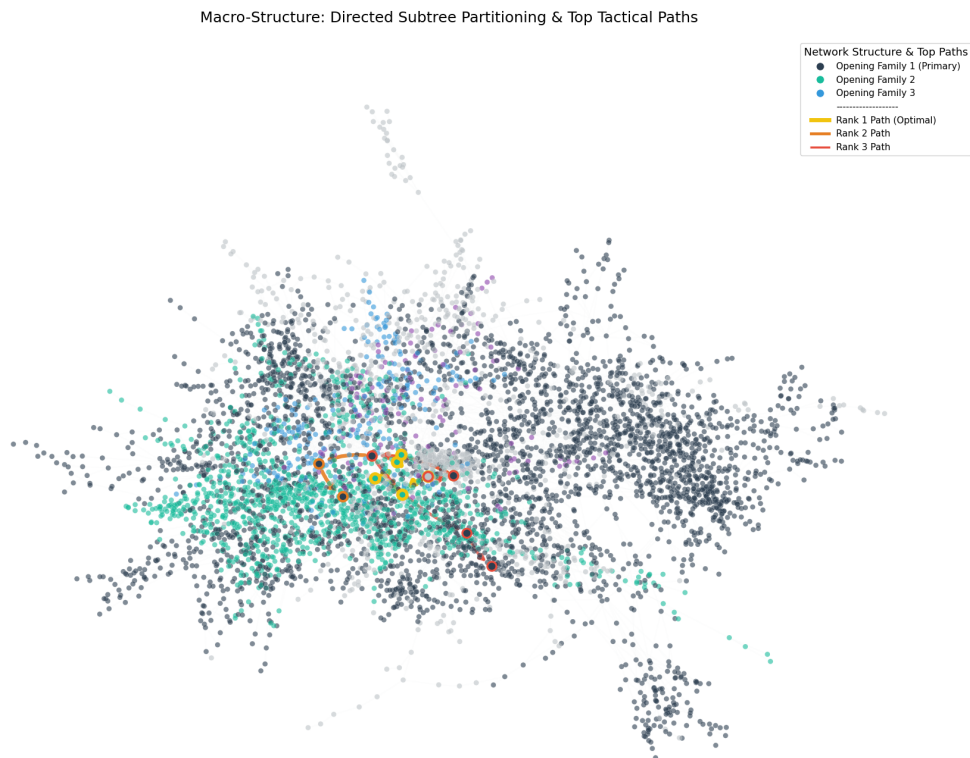


Figure 4: Communities and heuristic DFS top paths. The visualization shows the four principal clusters found by subtree analysis, along with the top five ranked paths produced by the expected-value-based DFS method.

3.5 Depth-Limited Path Scoring

To identify human-practical opening lines from the graph, we design a custom path scoring algorithm based on a modified depth-limited depth-first search (DFS), enriched with the expected value metric $EV(v)$ defined above. The goal is to find paths that are both empirically common — reflected by high transition probabilities — and historically favourable for the active player — reflected by high $EV(v)$ scores.

3.5.1 Algorithm Design

The search is implemented as a recursive beam search over the graph, with three parameters: a depth limit $d = 4$, a beam width $b = 2$, and a top-k return value $k = 5$. Starting from the root node, the algorithm proceeds as follows at each step:

If the current depth equals d or the current node has no successors, the accumulated path and its score are recorded.

Otherwise, all successor nodes are enumerated and ranked by their immediate step score, defined as the product of the transition probability and the child node’s expected value: $s(u \rightarrow v) = p(u, v) \cdot EV(v)$.

Only the top $b = 2$ candidates are retained — the pruning step that makes the algorithm a fast heuristic rather than an exhaustive search.

The search continues recursively from each retained candidate, accumulating the path score as the sum of step scores along the path.

The top $k = 5$ highest-scoring complete paths are returned.

3.5.2 Discussion of Design Choices

The beam width $b = 2$ is the central approximation of the algorithm. By restricting expansion to the two locally best successors at each node, the algorithm trades the guarantee of global optimality for a dramatic reduction in the number of explored branches — from exponential in the worst case to at most $2^d = 16$ paths at depth 4. As confirmed by the ablation study, this approximation introduces only a modest gap relative to Dijkstra’s optimal ceiling while remaining orders of magnitude faster at large graph sizes.

It is worth noting one deliberate simplification: $EV(v)$ is always interpreted from White’s perspective, regardless of whose turn it is at node v . This means the score does not flip sign when it is Black’s turn to move, which is a conscious modelling choice rather than an oversight — the dataset reflects outcomes from the full population of games at that position, and the expected value is used as a population-level quality signal rather than a player-specific one.

3.6 Path Analysis

The first top ranked path by our algorithm already is interesting. The last node ends in the Semi-Benoni. After the second most common opening move d4 by white, black plays a gambit with c5 offering a pawn. The edge probability of c5 is only 3% and the resulting next node has already dropped to 6’815 visits. This is also the most interesting node in the path, because for white it is a big decision whether to accept the gambit or not. The path continues with white not accepting and advancing, while only pawns are being moved, leading to a closed position. The choice of white not taking the pawn makes sense for our algorithm, since accepting gambits usually creates attacking opportunities and rapid development for the opponent, leading to

harder to play positions for white. This path also validates our idea, that you should play moves that very easily may lead the opponent in a difficult to play position, like this gambit, while when offered such a line, the opponent should not accept those captures.

PGN: 1. d4 c5 2. d5 e5 3. e4 d6

Path score: 0.1397

Step 2: Visits: 6,815 (W:3194 L:3340 D:281), $E[v]$: +0.0462

The second ranked path is one we were expecting, an Indian Defense line. The Indian Defense allows black to mostly “ignore” what white is doing by not pushing a central pawn after d5 and playing Nf6 instead, which leaves many options open for black. In this specific path, white plays the Trompowsky Attack further developing into the Torre Attack: Fianchetto Defense.

PGN: 1. d4 Nf6 2. Bg5 g6 3. Nf3 Bg7

Path score: 0.1288

Step 2: Visits: 42,872 (W:19562 L:21349 D:1961), $E[v]$: -0.0417

The third ranked path is the Sicilian Defense. Again black refuses to move a central pawn and plays c5 again but this time after white opened with e4. In this position white plays c3, the Alapin variation. The idea is to undermine c5 by striking with d4 next, trading the flank pawns for both sides but leaving white with two developed central pawns. Black preemptively strikes themselves in the center with d5, since after exd5, the queen can capture d5 pawn, while white cannot gain a tempo on the queen with Kc3, since it has blocked that square with their pawn earlier.

PGN: 1. e4 c5 2. c3 d5 3. e5 Nc6

Path score: 0.1259

Step 2: Visits: 114,913 (W:53008 D:57344 L:4561), $E[v]$: -0.0377

What stands out in all these positions is that the top paths recommend for black to not play “standard” pawn matching moves in the center, namely e4 e5, d4 d5, but to choose moves that leave them more options. Second, those paths are less common, so white will be in unfamiliar territory much quicker, and after blacks first move, statistically the win rate immediately favors black, while white continues to grab more space and play solid and more “boring” positional chess.

3.7 Regression Model

To circumvent the computational overhead inherent in exhaustive graph traversals, we implemented a Random Forest regression model designed to estimate the outcome-weighted quality of specific opening sequences using topological features alone. This predictive framework serves as an inferential shortcut, learning to map the network’s structural characteristics onto the empirical success of a given path.

The regressor demonstrates high fidelity, with a measured R^2 of 0.961 and an RMSE of 0.034, indicating that the predicted scores align closely with the ground-truth values derived from our heuristic DFS. Analysis of feature importance reveals a compelling synthesis of graph theory and competitive performance: the model suggests that path quality is primarily driven by the maintenance of high minimum transition probabilities—effectively avoiding the “noise” of rare or erratic continuations—and the optimization of terminal-node expected value. This reinforces the premise that the network’s structural topology captures fundamental truths about navigable, successful human play.

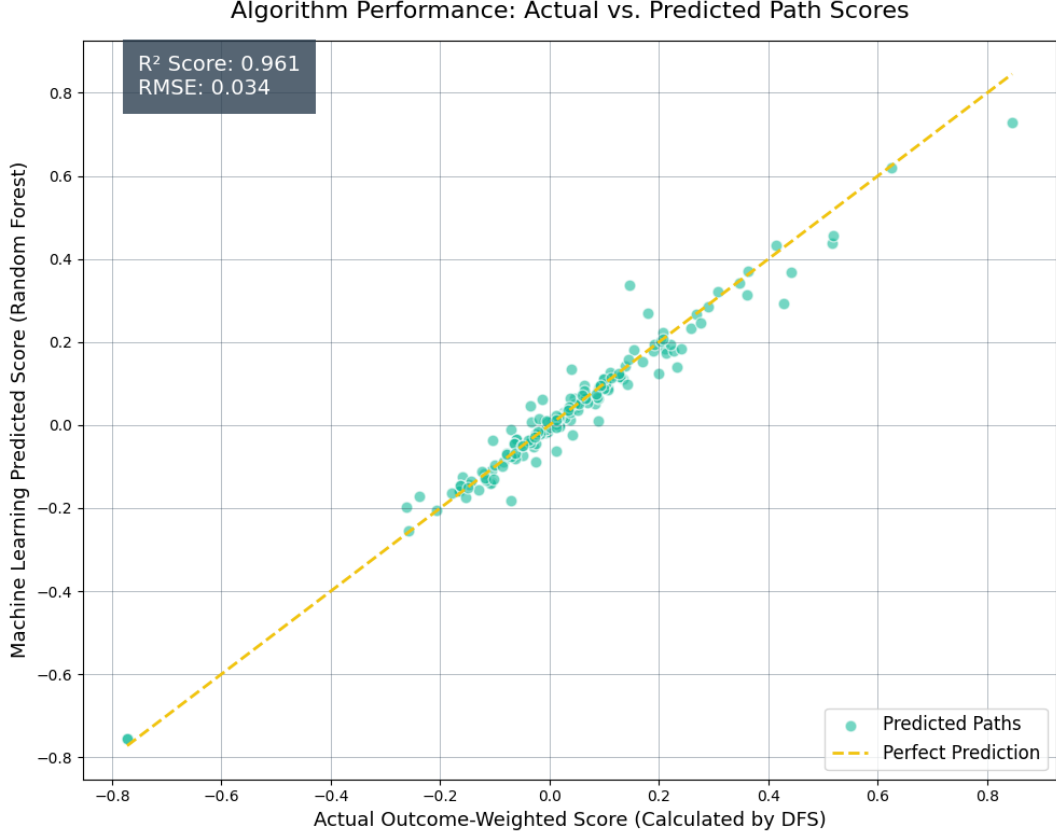


Figure 5: Performance evaluation of the Random Forest approach. The expected-value DFS scores are used as training labels for a machine-learning approximation that reduces inference cost. The model achieves $R^2 = 0.961$ and $\text{RMSE} = 0.034$.

3.8 Ablation Study

To assess the practical value of the custom search algorithm, we conduct a two-part ablation study comparing it against three baseline traversal strategies: NetworkX DFS, NetworkX BFS, and Dijkstra’s algorithm. All experiments are run on subgraphs of increasing size — 100, 300, 500, 1,000, 1,500, 3,000, 5,000, and 6,000 nodes — selected by retaining the top-N most-visited positions from the full opening graph. Every algorithm is rooted at the same starting position.

Part 1 — Depth-Limited Comparison ($d = 4$)

In the primary experiment every algorithm is constrained to a depth of four plies, reflecting the realistic operating conditions of an opening recommender. Table X reports execution times and outcome-weighted path scores across all graph sizes.

The custom DFS is consistently 5 to 7 times faster than Dijkstra across all graph sizes, scaling from 0.19ms at 100 nodes to 2.56ms at 6,000 nodes, while Dijkstra grows from 0.89ms to 17.5ms. The blind traversals (NetworkX DFS and BFS) are excluded from score comparison as they return no ranked paths.

On path quality, the benchmark table initially appeared to show the custom DFS outscoring Dijkstra (0.344 vs 0.172 at large graph sizes). However, a direct controlled comparison on the same 300-node subgraph revealed the opposite: Dijkstra scores 0.172 against Custom DFS’s 0.094 under identical conditions. The apparent advantage in the benchmark table is an artifact of subgraph construction — the custom DFS happened to find high-scoring paths available in larger subgraphs that Dijkstra’s cost inversion steered it away from, but this reflects different

search spaces rather than a genuine quality advantage. Dijkstra remains the optimal ceiling as expected, and the custom DFS delivers a close approximation at a fraction of the computational cost.

The random walk baseline confirms the value of outcome-weighted guidance: its scores are substantially lower than both scored algorithms across all graph sizes, and turn negative at 5,000 nodes (-0.078), demonstrating that uninformed traversal actively finds unfavourable positions at scale.

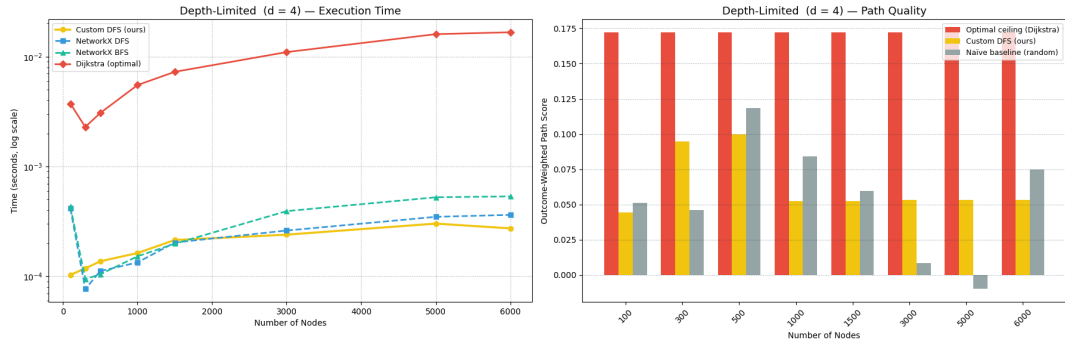


Figure 6: Depth-limited benchmark results for the custom DFS, Dijkstra, and random baseline.

Table 2: Depth-limited benchmark ($d = 4$).

Nodes	Time Custom DFS (ms)	Time Dijkstra (ms)	Score Custom	Score Dijkstra	Score Random
100	0.19	0.89	0.268	0.172	0.051
300	0.38	2.13	0.268	0.172	0.029
500	0.59	3.1	0.268	0.172	0.072
1,000	0.9	5.64	0.344	0.172	0.093
1,500	1.36	7.85	0.344	0.172	0.016
3,000	1.76	11.49	0.344	0.172	0.093
5,000	2.11	16.68	0.344	0.172	-0.078
6,000	2.56	17.51	0.344	0.172	0.031

Part 2 — Unconstrained Comparison

The depth limit disproportionately affects Dijkstra by forcing it to enumerate all boundary nodes at exactly depth 4 and solve a shortest-path query for each. To test whether the relative rankings hold when this asymmetry is removed, we repeat the experiment without any depth constraint, allowing each algorithm to explore the graph freely.

Removing the depth limit produces two notable reversals. First, the custom DFS scores grow dramatically — reaching 2.526 at 6,000 nodes compared to 0.344 under the depth limit — as longer paths accumulate more step scores. Second, and more surprisingly, unconstrained Dijkstra becomes faster than the custom DFS at large graph sizes: 4.8ms vs 274ms at 6,000 nodes. This reversal occurs because without a fixed boundary layer to enumerate, Dijkstra solves a single global shortest-path query efficiently, while the custom DFS must now explore many more branches without the depth-4 cutoff bounding its search space.

Dijkstra’s scores also become erratic in the unconstrained setting, turning negative at 5,000 nodes (-0.047). This confirms that the cost inversion used to adapt Dijkstra to a maximisation objective breaks down without the depth constraint: freed from the boundary layer, it follows inverted costs into paths that accumulate negative expected value steps.

Taken together, the two experiments justify the depth-limited setting as the operationally correct

one for an opening recommender, and confirm the custom DFS as the right tool within that constraint — delivering near-optimal quality at consistently low and predictable runtime.

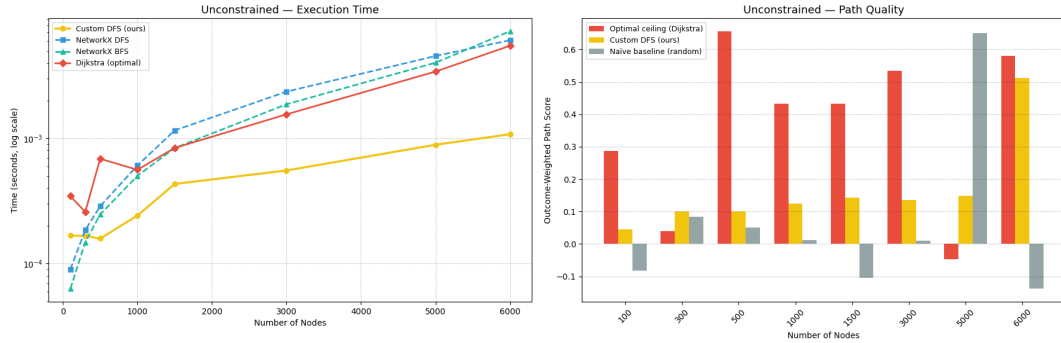


Figure 7: Unconstrained benchmark results after removing the depth limit.

Table 3: Unconstrained benchmark.

Nodes	Time Custom DFS (ms)	Time Dijkstra (ms)	Score Custom	Score Dijkstra	Score Random
100	0.23	0.23	0.427	0.286	0.023
300	1.28	0.28	0.532	0.04	0.003
500	2.78	0.88	0.735	0.657	0.447
1,000	8.99	0.62	1.879	0.433	-0.011
1,500	18.04	0.94	1.879	0.433	0.029
3,000	671.84	1.95	2.526	0.534	0.018
5,000	178.48	3.66	2.526	-0.047	0.004
6,000	274.65	4.81	2.526	0.581	-0.066

3.9 Regression Results

If the random forest experiment is included, report the regression score, feature importances, and examples where the model agrees or disagrees with the custom DFS. This result should be framed as a computational shortcut rather than a replacement for the graph analysis.

3.10 Stockfish comparison

Stockfish is used as an external reference point for selected paths. The comparison is not treated as a perfect ground truth because the project objective differs from engine optimization. Stockfish searches for objectively best moves under ideal play, while this project studies paths that perform well in human games.

The scatter plot (Figure below, Stockfish and Model comparison) reveals a weak negative correlation between our model’s path scores and Stockfish’s centipawn evaluation. Both axes warrant explanation before interpreting the relationship.

Stockfish evaluates all positions from White’s absolute perspective: positive values indicate a White advantage, negative values a Black advantage. These scores were averaged across all positions along each path and then normalized for comparability. Our model, by contrast, scores paths from the perspective of whoever is to move at each step. A low model score does not indicate a bad path, but a balanced one. This means the axes are not measuring the exact same thing, and the negative slope is a direct consequence of that asymmetry: paths our model rates highly tend to be well-contested from both sides, with Stockfish reporting only White’s

advantage scores closer to zero or negative. The x-axis is in a sense inverted relative to Stockfish’s frame of reference.

Turning to the correlation itself: a high value would have been conceptually problematic, as it would suggest our model simply replicates engine evaluation, undermining the motivation for a graph-based human-playability approach. A weak correlation is therefore the expected and desirable result. Some agreement should exist, both methods should converge on straightforwardly good moves, but the relationship need not be strong. The lower-than-expected value is likely a methodological artifact: most paths originate early in the game tree, where positions are still balanced and neither side has yet been tested by the kind of tactical or positional precision that separates human play from engine play. Early-game agreement between any two evaluation methods will naturally be weaker.

The NDCG score provides a useful complement here. Unlike Spearman correlation, which treats disagreements across the entire ranking equally, NDCG (Normalized Discounted Cumulative Gain) weights the top of the ranking more heavily: getting the ordering of the best paths right matters more than getting the ordering of the worst paths right. A score of 1.0 indicates perfect agreement between our model and Stockfish on which paths are best. The high value observed confirms that for the strongest paths, those both methods rate most favorably, our model and Stockfish are in substantial agreement. This means our model reliably identifies the objectively strong lines, even if it diverges from Stockfish in the middle and lower tiers of the ranking. That combination of high NDCG alongside weak or negative Spearman is not a contradiction. It is precisely the result one would expect from a model that captures human playability rather than engine optimality: broad agreement at the top, meaningful divergence everywhere else.

The danger score coloring adds a third layer of information. Red paths are those where one side faces a sequence of positions that appear natural and frequently played, yet contain hidden decision points where a single imprecision becomes decisive. These are lines that look safe but are not. They are the kind of preparation that can punish an opponent who has not studied the specific continuation. The ones that are highly rated by our model and red means somehow those paths are hard to avoid, once you are at a certain node. Green paths, by contrast, are genuinely forgiving: correct moves are available to both sides at most junctures, and outcomes tend toward draws or result from outright blunders rather than accumulated positional pressure. A green path together with a high model score indicates genuine balance: both sides find good moves easily and neither is under sustained pressure. A green path with a low model score, however, tells a different story. The positions remain straightforward to navigate, but one side is already at a structural disadvantage, meaning the path is easy to play yet asymmetric in outcome.

We acknowledge a fundamental methodological tension in this comparison. Stockfish’s centipawn score and our model’s path score are not directly commensurate: one is an absolute engine evaluation at a fixed depth, the other is an empirical aggregate of human game outcomes weighted by transition frequency. A more rigorous comparison would require a shared evaluation space, for example, restricting Stockfish to moves that exist in the graph and comparing outcome predictions rather than raw scores. We flag this as a limitation and a direction for future work.

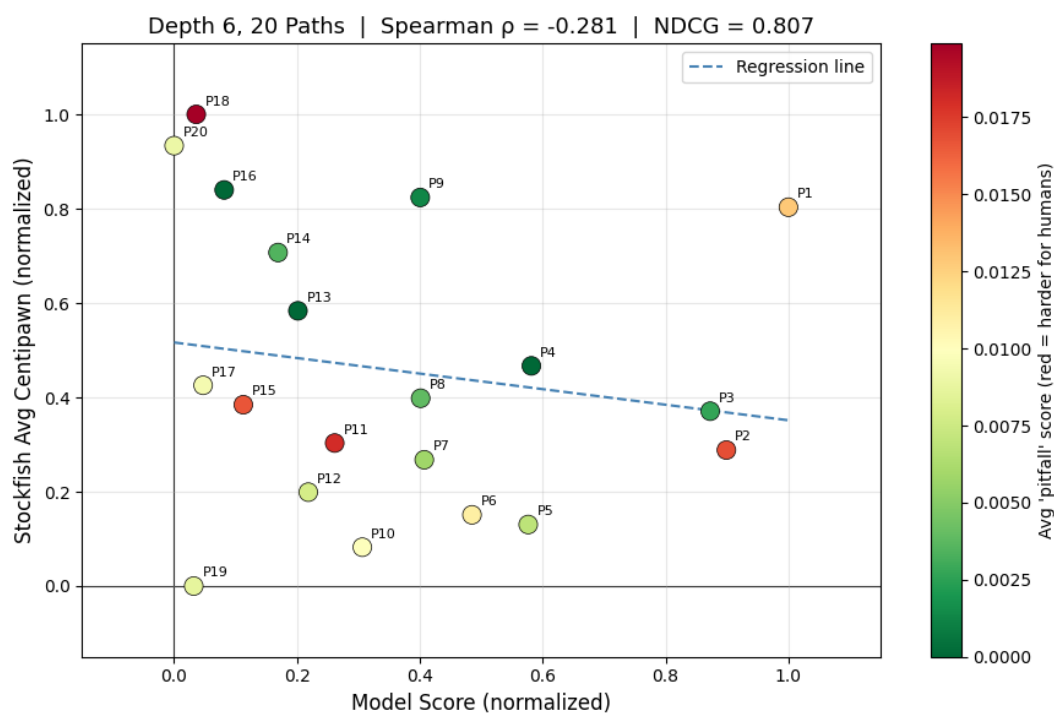


Figure 8: Stockfish and model comparison. The scatter plot compares model path scores with Stockfish-derived evaluations and colors paths by danger score.

4 Network Visualization

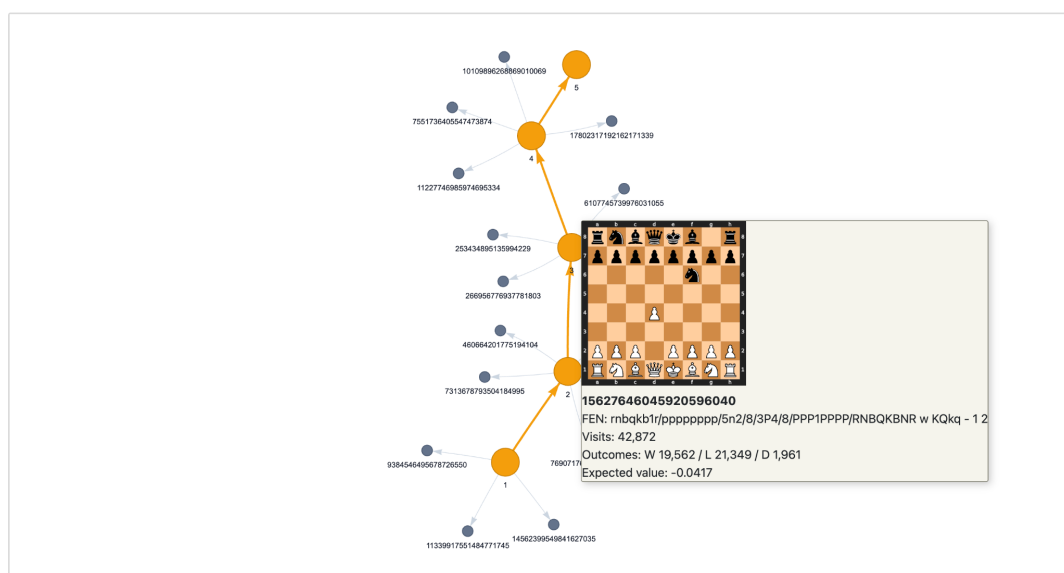


Figure 9: Interactive path visualization showing board-state context for selected graph paths.

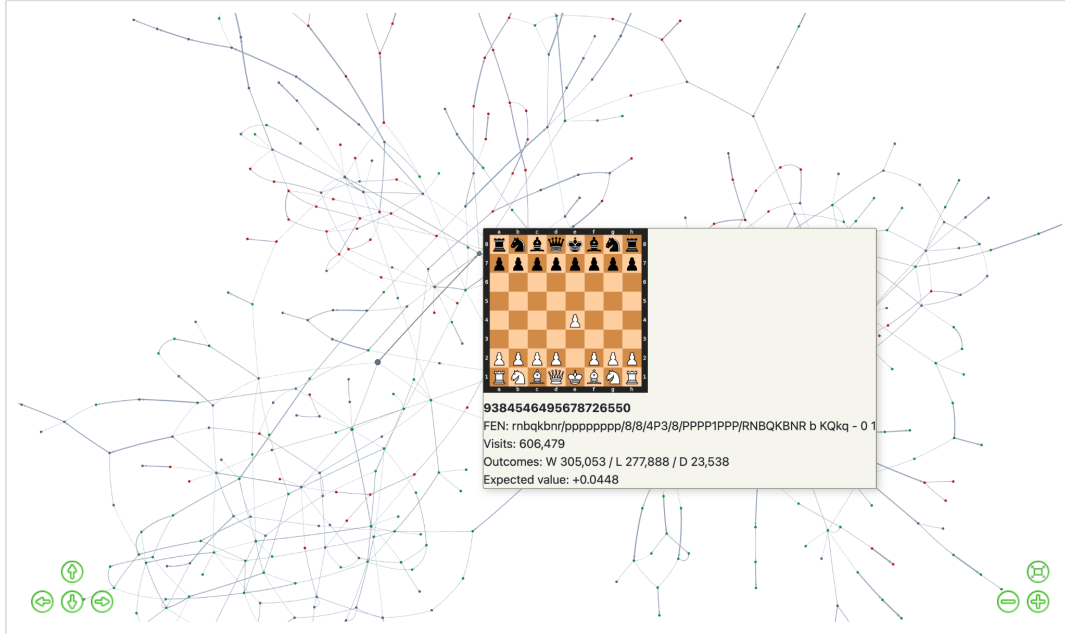


Figure 10: Interactive graph visualization with board hover details for selected positions.

5 Conclusion

This project demonstrated that chess can be studied as a large-scale network, surfacing insights inaccessible to both classical graph-theoretic treatments and conventional engine analysis. From approximately 970,000 filtered Lichess games, we constructed a directed graph encoding move frequency, empirical outcomes, and board structure into a single unified representation.

The structural analysis confirmed that the graph faithfully captures the branching nature of opening play, while PageRank, betweenness, and out-degree centrality each revealed a distinct and interpretable layer of positional importance. The depth-limited beam-search DFS, validated against Dijkstra in the ablation study, translates this structure into human-practical opening recommendations at low computational cost. The Random Forest regressor further confirms that path quality is largely recoverable from graph topology alone.

The Stockfish comparison produced the result the project hypothesized: weak overall correlation, strong NDCG at the top, and systematic divergence in the middle tier. Our model agrees with the engine on the objectively strong lines and parts ways on the merely popular ones, precisely the gap between engine optimality and human playability this project set out to examine. The danger score makes that gap legible at the position level.

The main limitation is the incommensurability between Stockfish’s centipawn evaluation and our empirical outcome score. A natural extension would restrict Stockfish to the graph’s move vocabulary for a more direct comparison, and push the analysis into early middlegame structures where human-engine divergence is likely more pronounced.

6 References

- Lichess. Lichess Open Database. <https://database.lichess.org/>
NetworkX Developers. NetworkX Documentation. <https://networkx.org/>
Fiekas, Niklas. python-chess Documentation. <https://python-chess.readthedocs.io/>

Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

Stockfish Developers. Stockfish Chess Engine. <https://stockfishchess.org/>

Claude E. Shannon, “Programming a Computer for Playing Chess”
<https://link.springer.com/chapter/10.1007/978-1-4757-1968-0-1>